

Lab: 0011

Combinational Circuits

From Boolean minimisation to the data-selector circuits that route every bus in a modern CPU.

CSA222: Modern Computer Architecture

Where We Left Off

Lab 1 built the gates. Lecture 2 shrank them. Today we build the circuits that pick between them.

Lab 1

Seven gates in Verilog — AND, OR, NOT, XOR, NAND, NOR, XNOR — plus a self-checking testbench.

Lab 2

SOP / POS, Karnaugh maps, and the proof that NAND (or NOR) alone can build any circuit.

Lab 3 — Today

Multiplexers and demultiplexers: the gate patterns that select and route data — the ternary operator, the case statement, and structural instantiation.

What You'll Build Today

By the end of this lab, you'll have four data-selector circuits running in simulation.

2:1 MUX

Two inputs, one select line

4:1 MUX

Four inputs, two select lines

1:2 DEMUX

One input routed to one of two outputs

1:4 DEMUX

One input routed to one of four outputs

Naming convention: an n:1 MUX has n inputs. A demultiplexer is the mirror image, so this course names it by its 1:n output count — 1:2 and 1:4 — to avoid an "n:1 demux" that would read backwards.

Behavioural Modeling

Procedural blocks — and the single most-tested rule in this entire lecture.

The initial Block

Runs exactly once, starting at simulation time zero — testbench territory.

INITIAL — RUNS ONCE

```
initial begin
```

```
  a = 0; b = 0;
```

```
  #10 a = 0; b = 1;
```

```
  #10 $finish;
```

```
end
```

A module can contain more than one initial block — each starts independently at t=0 and runs to completion (or the next timing control) concurrently with the others.

The always Block

Re-triggers every time its sensitivity list changes — the procedural equivalent of assign.

always @(*) — combinational

Re-evaluates whenever any signal read inside the block changes. The (*) auto-infers the sensitivity list — the modern, error-proof way to write it.

```
always @(*) y = a & b;
```

always @(posedge clk) — sequential

Re-evaluates only on the rising edge of clk. This is how registers and flip-flops are described — a full preview, properly covered in Module 2.

```
always @(posedge clk) q <= d;
```

The left-hand side inside any always block must be declared reg — this is Part III's rule, now in action.

``timescale`, ``define`, ``include`

Three directives you'll see at the top of nearly every file this course.

``timescale`

Sets the simulation time unit and precision. Every Lab 1 file starts with this exact line.

```
`timescale 1ns/1ps
```

``define`

A text macro, substituted before compilation — like `#define` in C. Used sparingly; parameter is usually the better tool.

```
`define WIDTH 8
```

``include`

Splices another file's text in verbatim at compile time — how large designs share common definitions across files.

```
`include "defs.v"
```

Testbench Theory

The constructs behind Lab 1's testbench — explained, not yet typed.

Why a Testbench Has No Ports

The one module in your entire design that connects to nothing but simulation itself.



Nothing outside simulation ever instantiates it

Every other module in this course gets instantiated by something — `gates_top` instantiates `and_gate`; a real chip's top-level module gets synthesized and fabricated. A testbench is instantiated by nobody. It sits at the very top of the hierarchy purely to drive stimulus into the design under test (DUT) and observe what comes back — so it has no reason to expose a port list at all.

\$time, \$finish, \$stop

Controlling and reading the simulation clock itself.

\$time

Returns the current simulation time — what powers the `t=%0t` field in every `$monitor` line you've seen.

\$finish

Ends the simulation entirely and exits the simulator. This is how Lab 1's testbench cleanly stops after its stimulus sequence.

\$stop

Pauses simulation without exiting — hands control back to an interactive simulator session. Rare in scripted, batch-run testbenches.

\$dumpfile & \$dumpvars

The two lines that turn a silent simulation into a viewable waveform.

ENABLING WAVEFORM CAPTURE

```
$dumpfile("gates_tb.vcd"); // names the output file  
$dumpvars(0, gates_tb);   // records every signal from level 0 down
```

\$dumpfile alone creates an empty VCD — both lines are required. This is exactly the pair Lab 1's Common Mistakes slide warns about forgetting.

Generating Stimulus

initial plus #delay is the entire vocabulary needed to drive a DUT through every input combination.

A FOUR-COMBINATION STIMULUS SEQUENCE

```
initial begin
    a=0; b=0;
#10; a=0; b=1;
#10; a=1; b=0;
#10; a=1; b=1;
#10; $finish;
end
```

#10 between each line advances simulation time by 10 units before the next assignment — without it, every line would execute at t=0.

Verilog in 60 Seconds

Just enough to start. We'll come back to syntax in later labs.

module

Defines a hardware unit. Has input and output ports.

wire

A net — connects things. Used for combinational signals.

reg

A variable — needed for assignments inside `always` blocks.

assign

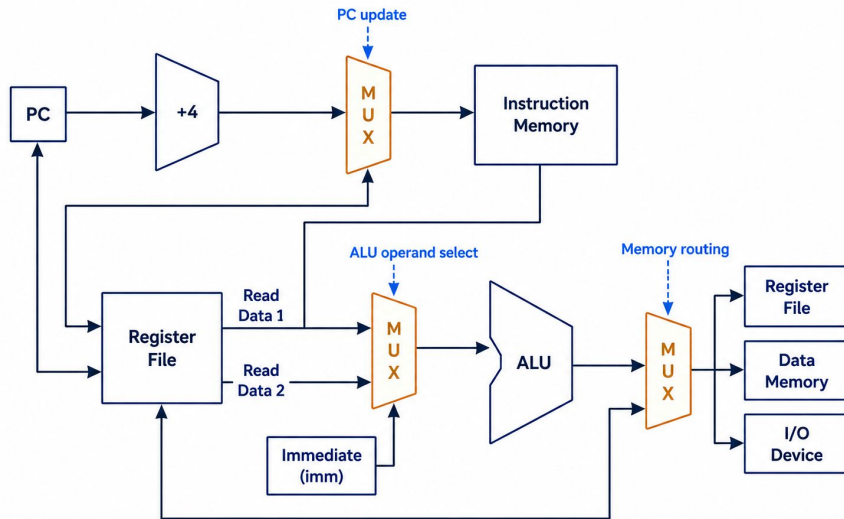
Continuous assignment. Use this for combinational logic.

always @(*)

Procedural block. Re-runs when ANY listed signal changes.

Why a Data Selector Circuit?

One circuit primitive shows up in almost every block of a CPU datapath.



- **ALU operand select**

Choosing between a register value and an immediate constant before the ALU.

- **PC update**

Choosing the next instruction address: PC+4, a branch target, or a jump target.

- **Memory routing**

Sending one data bus to one of several destinations — registers, cache, or I/O.

Every one of these is either a multiplexer (many signals → one) or a demultiplexer (one signal → many).

Part 1 • Multiplexers

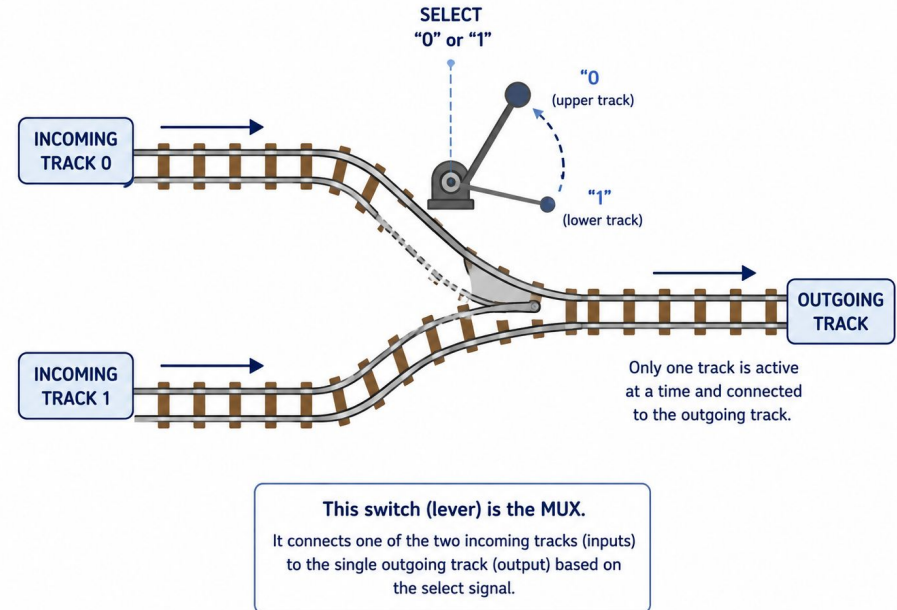
Many signals in. One signal out — chosen by a select line.

What Is a Multiplexer?

A digital switch: it selects exactly one of several inputs and routes it to a single output.

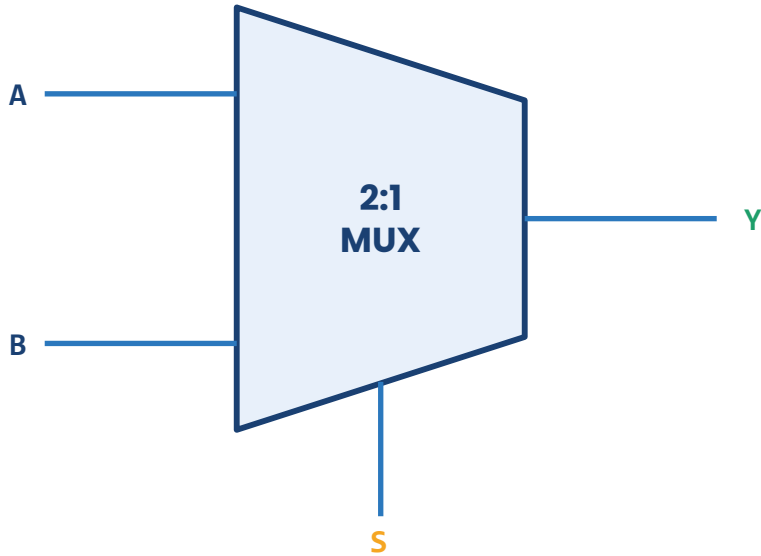
MUX = Multiplexer = Data Selector

- ✦ A MUX with n select lines can choose among 2^n inputs.
- ✦ 1 select line $\rightarrow$ 2 inputs (2:1 MUX).
- ✦ 2 select lines $\rightarrow$ 4 inputs (4:1 MUX).
- ✦ Only one input ever reaches the output at a time — the rest are ignored.



2:1 MUX • Truth Table

One select line S chooses between I0 and I1.



$$S = 0 \rightarrow Y = A$$

$$S = 1 \rightarrow Y = B$$

S	Y
0	I0
1	I1

“if S is 0, pass I0; if S is 1, pass I1.” Exactly one product term survives for any value of S — that’s what makes it a selector and not just a logic function.

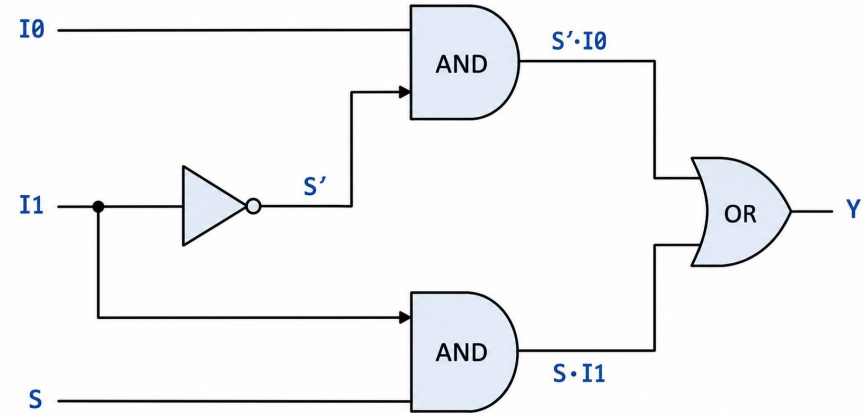
2:1 MUX • Expression & From Equation to Gates

Two AND gates, one OR gate, one inverter — the equation built in silicon.

Boolean expression (sum-of-products form):

$$Y = S' \cdot I_0 + S \cdot I_1$$

This is exactly the SOP form Lecture 2 taught you to derive from a truth table — the MUX equation is just the K-map result written out.



Exercise 1 • Build a 2:1 MUX (Gate-Level)

Write the equation from Slide 7 directly as Verilog primitives.

```
mux2_1_gate.v
module mux2x1 (
    input  wire a, b, sel,
    output wire y
);

    wire sel_n, a0, a1;

    not  u0 (sel_n, sel);           // S'
    and  u1 (a0, a, sel_n);        // S'·I0
    and  u2 (a1, b, sel);          // S·I1
    or   u3 (y, a0, a1);           // sum

endmodule
```

KEY POINTS

sel_n

A named wire for the inverted select — no gate output is left anonymous.

not / and / or

The same primitive gates from Lab 1, just wired to a new pattern.

u0, u1...

Instance names are mandatory for primitive gates with more than one instance of the same type in a module.

New Construct • The Conditional Operator

A one-line hardware selector — the exact shape of a multiplexer.

syntax

```
assign y = condition ? value_if_1 : value_if_0;
```

? reads as “if” — evaluates **condition** first.

: reads as “else” — the fallback value.

This is a continuous assignment, so it belongs after **assign**, never inside an **always** block.

`sel ? in1 : in0` is a 2:1 MUX in eight characters. Chain them and you can build any MUX width — but as the chain grows, a case statement (next up) reads far more clearly.

Exercise 1 • Alternate: the Ternary Version

Same interface, one line of behavioral Verilog instead of four gates.

```
mux2_1_ternary.v
module mux2x1 (
    input  wire a, b, sel,
    output wire y
);

    assign y = sel ? b : a;

endmodule
```

WHY BOTH VERSIONS?

Gate-level

Shows exactly which transistors switch — this is what synthesis produces either way.

Ternary

Shows what you'll actually type once you trust the tool to do the gate mapping.

Same module name, same port list — pick one file at a time. Both simulate identically. From here on, this course defaults to the shorter behavioral style unless a lab specifically asks for gate-level, exactly like Lab 1's assign vs primitive distinction.

Exercise 1 • Testbench for the 2:1 MUX

Drive all four (i0, i1, sel) combinations and record the waveform.

`mux2_1_tb.v`

```
module mux2x1_tb;
    reg a, b, sel;
    wire y;
    mux2x1 uut (.a(a), .b(b), .sel(sel), .y(y)
    );
    initial begin
        sel = 0; a = 0; b = 0;
        #10 sel = 0; a = 0; b = 1;
        #10 sel = 0; a = 1; b = 0;
        #10 sel = 0; a = 1; b = 1;
        #10 sel = 1; a = 0; b = 0;
        #10 sel = 1; a = 0; b = 1;
        #10 sel = 1; a = 1; b = 0;
        #10 sel = 1; a = 1; b = 1;
        #10 $finish;
    end
    initial begin
        $dumpfile("mux2x1_wave.vcd");
        $dumpvars(0, mux2x1_tb);
        $monitor("Time = %0t | sel = %b, a = %b, b = %b | y = %b",
            $time, sel, a, b, y);
    end
endmodule
```

4:1 MUX • Truth Table

Two select lines now address four inputs — exactly 2^2 .

S1	S0	Y
0	0	I0
0	1	I1
1	0	I2
1	1	I3

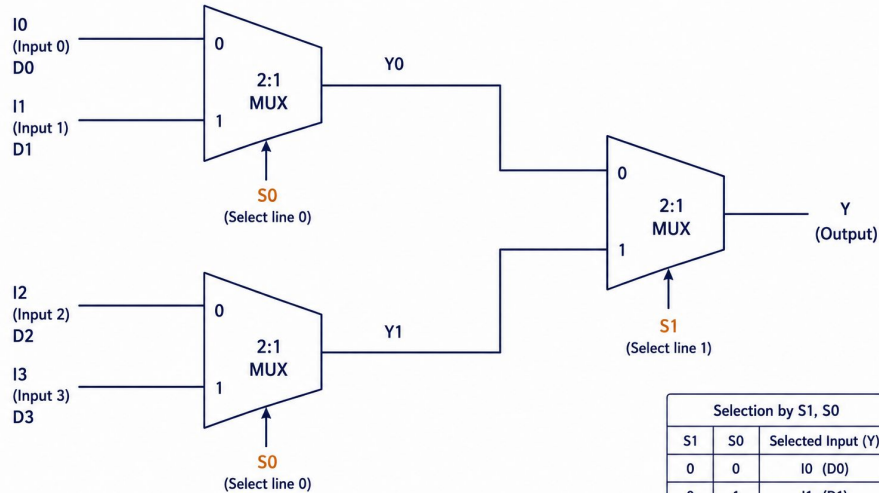
Boolean expression:

$$Y = S1'S0' \cdot I0 + S1'S0 \cdot I1 + S1S0' \cdot I2 + S1S0 \cdot I3$$

Four AND terms, one per input, each gated by the unique minterm of (S1, S0) that selects it — then OR'd together. Writing all four terms out by hand is exactly why Lecture 2's SOP-from-truth-table method matters once a circuit outgrows two inputs.

4:1 MUX • Two Ways to Build It

Structural composition vs. behavioral description — same circuit, two mindsets.



Selection by S1, S0		
S1	S0	Selected Input (Y)
0	0	I0 (D0)
0	1	I1 (D1)
1	0	I2 (D2)
1	1	I3 (D3)

Structural

Instantiate the mux2_1 module three times and wire them into a tree. Mirrors Lab 1's top.v — building bigger circuits out of smaller, already-verified ones.

Behavioral

Describe the truth table directly with a case statement inside an always block. Faster to write, and how you'll build most real designs.

New Construct • The case Statement

A readable, self-documenting way to describe a truth table inside an always block.

syntax

```
always @(*) begin
  case (sel)
    2'b00: y = i0;
    2'b01: y = i1;
    2'b10: y = i2;
    default: y = i3;    // covers 2'b11 AND catches X/Z
  endcase
end
```

y must be declared **reg**, not **wire** — it's assigned inside a procedural block. **default** is not optional: an incomplete case list infers a latch instead of a MUX — the single most common bug in this exercise.

Exercise 2 · 4:1 MUX — Behavioral Build

The same interface, described with a case statement.

```
mux4_1_behavioral.v
module mux4_1 (
    input wire i0, i1, i2, i3,
    input wire s0, s1,
    output reg y
);

always @(*) begin
    case ({s1, s0})
        2'b00: y = i0;
        2'b01: y = i1;
        2'b10: y = i2;
        default: y = i3;
    endcase
end
endmodule
```

Exercise 2 · Testbench — Sweep Every Select Value

New construct: a for loop generates all four select combinations automatically.

```
mux4_1_tb.v
module mux4_1_tb;
    reg i0, i1, i2, i3, s0, s1;
    wire y;
    integer k;

    mux4_1 dut (.i0(i0), .i1(i1), .i2(i2), .i3(i3), .s0(s0), .s1(s1), .y(y));

    initial begin
        $dumpfile("mux4_1_tb.vcd"); $dumpvars(0, mux4_1_tb);
        i0=0; i1=1; i2=0; i3=1;
        for (k = 0; k < 4; k = k + 1) begin
            {s1, s0} = k; #10;    // walks 00, 01, 10, 11
        end
        $finish;
    end
endmodule
```

New Construct • for Loops in a Testbench

Stop hand-typing every input combination — let the simulator count for you.

```
integer k;
```

Declares a variable to hold the loop counter — testbench-only, never in synthesizable logic.

```
for (k=0; k<4; k=k+1)
```

Same three-part syntax as C: init; condition; step.

```
{s1, s0} = k;
```

Concatenation on the left-hand side — splits k's two low bits straight into the two select wires.

```
#10 inside the loop
```

Advances simulation time on every iteration, so each combination gets its own visible slice of the waveform.

Part 2 • Demultiplexers

One signal in. Many possible outputs — routed by a select line.

What Is a Demultiplexer?

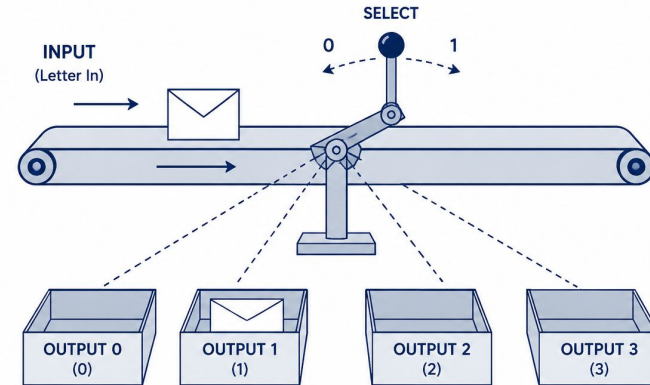
The mirror image of a MUX: one input, routed to exactly one of several outputs.

- ✦ DEMUX = a data distributor, the reverse of a data selector.
- ✦ n select lines route the input to one of 2^n outputs.
- ✦ Every output not selected is driven to 0 — nothing is left floating.

Naming note: “2:1 DEMUX” would literally mean 2 inputs $\rightarrow$ 1 output — backwards for what we're building. This course calls these circuits 1:2 and 1:4 DEMUX (1 input : n outputs) instead.

MAIL SORTING ANALOGY FOR A DEMUX

A letter on the conveyor belt is directed by a mechanical gate into one of the output mail boxes based on the SELECT lever.



SELECT determines which output mail box receives the letter.

SELECT	ACTIVE OUTPUT
0	$\rightarrow$ Output 0
1	$\rightarrow$ Output 1
2	$\rightarrow$ Output 2
3	$\rightarrow$ Output 3

NOTE Only the selected output mail box receives the letter; all other mail boxes remain empty.

1:2 DEMUX • Truth Table & Expression

One select line S routes input D to either Y0 or Y1.

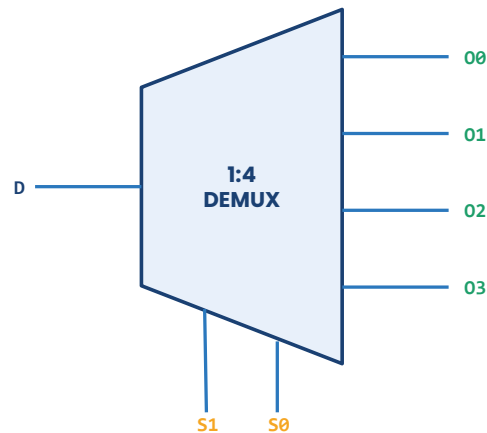
S	Y0	Y1
0	D	0
1	0	D

Boolean expressions:

$$Y0 = S' \cdot D$$

$$Y1 = S \cdot D$$

Notice the symmetry with the 2:1 MUX equation on Slide 7 — same AND/inverter pattern, just fanning one input out to two gated outputs instead of gating two inputs into one.



Exercise 3 · Build a 1:2 DEMUX

Two AND gates and an inverter — the mirror image of Exercise 1.

```
demux1_2.v
module demux1_2 (
    input  wire i, sel,
    output wire y0, y1
);

    assign y0 = ~sel & i;
    assign y1 =  sel & i;

endmodule
```

No case statement needed here — with only one select bit and two mutually exclusive outputs, two continuous assignments are already as clear as it gets.

Exercise 3 · Testbench for the 1:2 DEMUX

Confirm the unselected output always reads 0 — the defining behavior of a DEMUX.

```
demux1_2_tb.v
module demux1_2_tb;
    reg i, sel;
    wire y0, y1;

    demux1_2 dut (.i(i), .sel(sel), .y0(y0), .y1(y1));

    initial begin
        $dumpfile("demux1_2_tb.vcd"); $dumpvars(0, demux1_2_tb);
        i=1; sel=0; #10;    // expect y0=1, y1=0
        i=1; sel=1; #10;    // expect y0=0, y1=1
        i=0; sel=1; #10;    // expect y0=0, y1=0
        $finish;
    end
endmodule
```

1:4 DEMUX • Truth Table

Two select lines route D to exactly one of four outputs; the rest read 0.

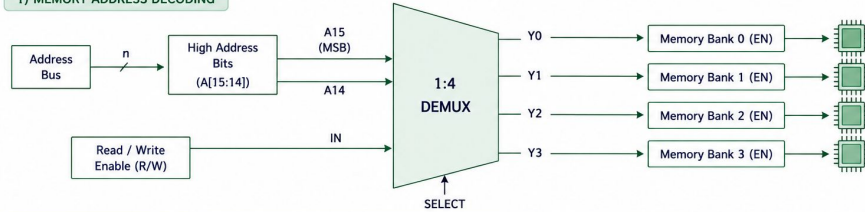
S1	S0	Y0	Y1	Y2	Y3
0	0	D	0	0	0
0	1	0	D	0	0
1	0	0	0	D	0
1	1	0	0	0	D

With a case statement, this table becomes almost literal Verilog — each row is one case branch that drives the whole output bus at once.

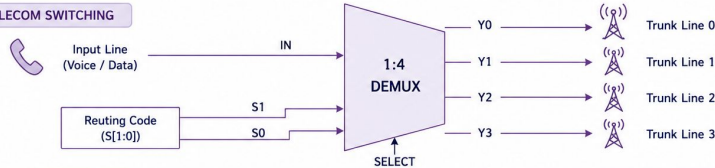
1:4 DEMUX • Where This Shows Up

Any time one resource must be shared by steering it to different destinations in turn.

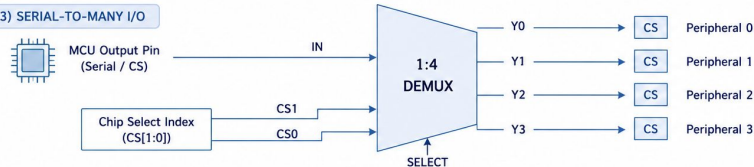
1) MEMORY ADDRESS DECODING



2) TELECOM SWITCHING



3) SERIAL-TO-MANY I/O



Memory address decoding

The high bits of an address 'demux' a read/write enable signal to exactly one memory bank or chip.

Telecom switching

A single voice/data line is routed to one of several trunk lines based on a routing code.

Serial-to-many I/O

One microcontroller output pin, time-shared across several peripheral chip-select lines.

Exercise 4 · Build a 1:4 DEMUX

A 4-bit output bus, driven entirely by a case statement.

```
demux1_4.v
module demux1_4 (
    input wire d,
    input wire [1:0] sel,
    output reg [3:0] y
);

always @(*) begin
    y = 4'b0000;           // clear all outputs first
    case (sel)
        2'b00: y[0] = d;
        2'b01: y[1] = d;
        2'b10: y[2] = d;
        default: y[3] = d;
    endcase
end
endmodule
```

Exercise 4 · Testbench for the 1:4 DEMUX

Reuse the for-loop pattern from Exercise 2 to sweep all four select values.

```
demux1_4_tb.v
module demux1_4_tb;
    reg d;
    reg [1:0] sel;
    wire [3:0] y;
    integer k;

    demux1_4 dut (.d(d), .sel(sel), .y(y));

    initial begin
        $dumpfile("demux1_4_tb.vcd"); $dumpvars(0, demux1_4_tb);
        d = 1;
        for (k = 0; k < 4; k = k + 1) begin sel = k; #10; end
        $finish;
    end
endmodule
```

Common Errors • And the Fix

Four mistakes that will eat 30 minutes if you don't recognize them.

Missing default in a case

Every select value not covered infers a latch, not a MUX. Always add default (or cover every combination explicitly).

y declared as wire but assigned in always Anything assigned inside always must be reg. Continuous assign targets must be wire.

Overlapping case items

Verilog case matches top-to-bottom and stops at the first hit — duplicate or missing bit patterns silently misbehave instead of erroring.

Select bus built from the wrong bit order

$\{s1, s0\} \neq \{s0, s1\}$. Double-check concatenation order against your truth table before simulating.

Take These Home

Three stretch problems. Solutions discussed at the start of Lab 4.

01

Build an 8:1 MUX

Combine two 4:1 MUXes and one 2:1 MUX (the third select bit) into an 8-input selector. Reuse Exercise 2's structural pattern.

02

Parameterize the MUX

Rewrite mux4_1 with a parameter WIDTH so i0–i3 and y become WIDTH-bit buses instead of single wires.

03

Add an enable input

Add an active-low enable_n to demux1_4 so all outputs read 0 whenever enable_n = 1, regardless of sel.



**Thanks
for
watching!**